EXP 6: Write a program to implement Naïve Bayes algorithm in python and to display the results using confusion matrix and accuracy.

CODING:

```
import math


# Training Dataset
X = [
    [1, 2], [2, 3], [2, 2], [3, 3],
    [6, 7], [7, 8], [8, 7], [7, 6],
    [1, 3], [3, 2], [6, 8], [8, 6]
]

Y = ['A', 'A', 'A', 'A', 'B', 'B',
     'B', 'B', 'A', 'A', 'B', 'B']

# Split Training and Testing Data
X_train = X[:4] + X[6:10]
Y_train = Y[:4] + Y[6:10]

X_test = X[4:6] + X[10:12]
Y_test = Y[4:6] + Y[10:12]

# Calculate Gaussian Probability
def gaussian(x, mean, variance):
    return (1 / math.sqrt(2 * math.pi * variance)) * math.exp(
        -((x - mean) ** 2) / (2 * variance)
    )
```

Scanned with OKEN Scanner

```
# Train Naive Bayes Model
def train(X, Y):
    model = {}

    for label in set(Y):
        rows = [X[i] for i in range(len(Y)) if Y[i] == label]

        mean = []
        variance = []

        for j in range(len(X[0])):
            values = [row[j] for row in rows]
            avg = sum(values) / len(values)

            var = sum((v - avg) ** 2 for v in values) / len(values)
            mean.append(avg)
            variance.append(max(var, 0.000001))

        prior = len(rows) / len(Y)
        model[label] = (prior, mean, variance)

    return model

# Predict Class
def predict(sample, model):
    probabilities = {}
```

Scanned with OKEN Scanner

```
for label, values in model.items():
    prior, mean, variance = values
    probability = math.log(prior)

    for i in range(len(sample)):
        p = gaussian(sample[i], mean[i], variance[i])
        probability += math.log(max(p, 1e-300))

    probabilities[label] = probability

    return max(probabilities, key=probabilities.get)

# Train the Model
model = train(X_train, Y_train)

# Test the Model
predictions = []

for sample in X_test:
    predictions.append(predict(sample, model))

# Calculate Accuracy
correct = 0

for i in range(len(Y_test)):
    if Y_test[i] == predictions[i]:
        correct += 1
```

Scanned with OKEN Scanner

```
accuracy = (correct / len(Y_test)) * 100

# Confusion Matrix
labels = sorted(set(Y_train))
matrix = []

for actual in labels:
    row = []
    for predicted in labels:
        count = sum(
            1 for i in range(len(Y_test))
            if Y_test[i] == actual and predictions[i] == predicted
        )
        row.append(count)
    matrix.append(row)

# Display Results
print("Actual Values:", Y_test)
print("Predicted Values:", predictions)

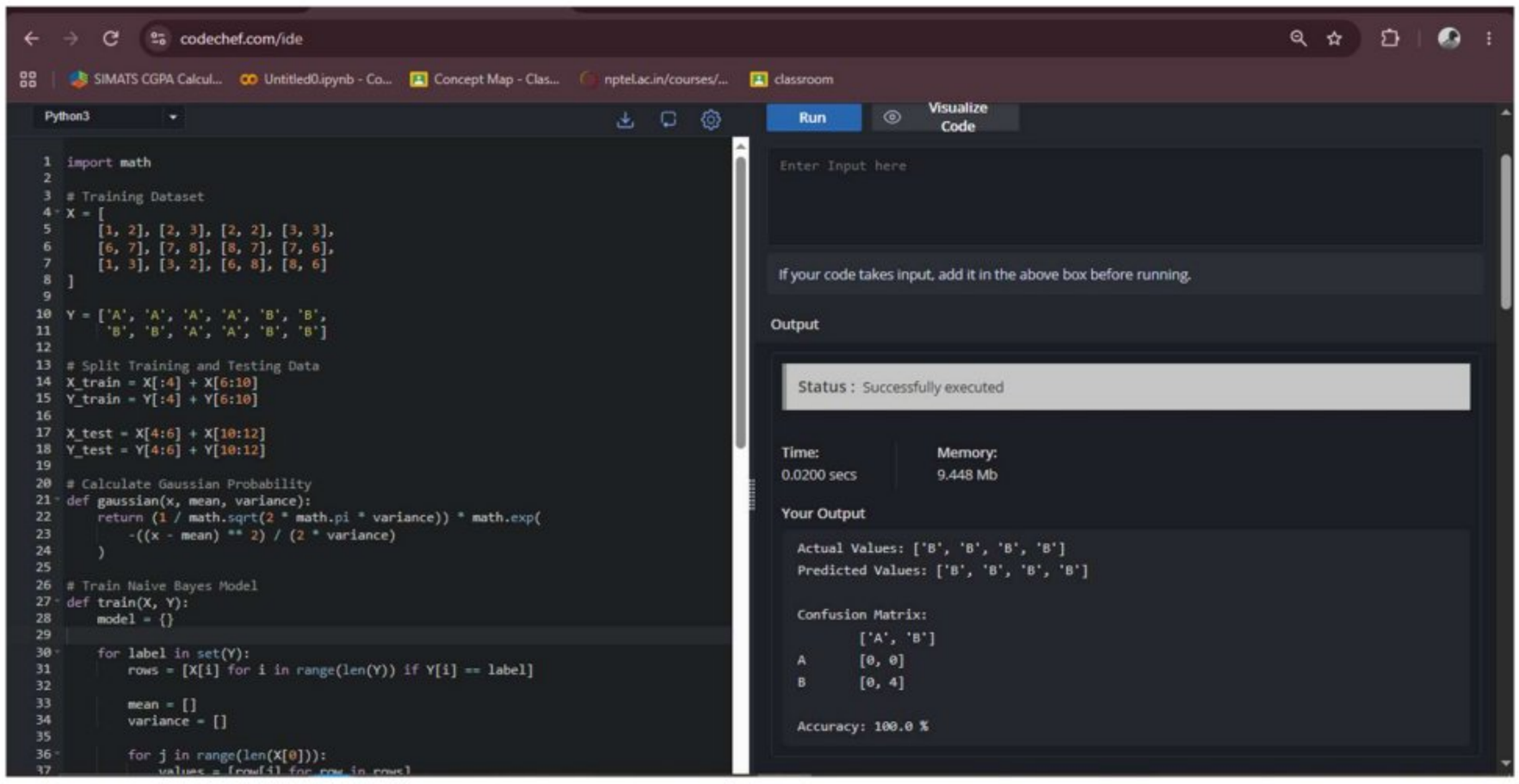
print("\nConfusion Matrix:")
print("      ", labels)

for i in range(len(labels)):
    print(labels[i], "    ", matrix[i])

print("\nAccuracy:", accuracy, "%")
```

Scanned with OKEN Scanner

OUTPUT:



Scanned with OKEN Scanner